



PROJECT DOCUMENTATION OF GRUBFLOW

Nayan Makanjee (2801261)

Hemesh Parshotam (2836244)

Jai Parbhoo (2823272)

Saurav Lall (2831658)

Procedures and Processes

The development of the Restaurant Order Tracker application focused on building a simple yet effective mobile platform to improve communication between takeaway restaurants and their customers. The app enhances order visibility for customers while giving staff a streamlined interface to manage and update order statuses. The system has two different user roles—customers and employees—each of which is chosen upon registration and has features specific to their needs.

Users are asked to log in or register when they first use the app. A registration page requiring the user's name, email address, password, and role selection must be filled out by new users. Validation ensures emails are well-formed and usernames are unique. Upon choosing a role, if the user is a staff member, they are required to enter the restaurant that they work at. Once validated, user data is saved to the Users table. Returning users log in with their credentials, which are matched against the same table.

Customer Flow

Upon login, customers are taken to their **Order History** page. This dashboard displays all the customer's past and current orders, each showing:

- The restaurant name
- Time of order
- Current status (Pending, Ready, or Collected)
- Payment state
- Thumbs up/down rating buttons

Orders are retrieved from the Orders table and filtered by the user's ID. If the user has no orders, a prompt encourages them to place one. After an order is collected, the customer may rate it with a simple thumbs up or down. This rating is saved in the Ratings table and tied to both the user and order. These ratings help calculate each restaurant's **average rating**, which is updated in real-time via backend logic.

Staff Flow

Staff members are directed to a page listing all orders tied to their restaurant where they can create a new order, update the status (Pending, Ready, Collected) and use the data to review performance.

To create an order, staff fill in a form that includes:

- Customer's name (validated against registered users)
- Restaurant name (auto-filled)
- Time of creation (captured automatically)

New orders are saved with a default status of "Pending" which the staff can later update to "**Ready**" or "**Collected**". These updates are saved in the Orders table and reflected on the customer's dashboard.

Staff can also view the rating of the restaurant they belong to in the staff menu, showing average customer ratings for their orders fulfilled. This is useful for monitoring service quality.

Additional Features and Session Handling

Staff users can **search for customers** and view their order histories. All communication between the app and backend is handled using **PHP scripts** that query a **MySQL** database and return data in JSON format for the app to display.

Users can **log out** of the app anytime using the toolbar, which returns them to the login screen. If a user enables the **Remember me** feature, their latest logged in information is stored automatically in the text boxes, ready for the user to press the log in button.

The Restaurant Order Tracker app separates staff and customer roles and provides a reliable and intuitive experience for both types of users.

Business Rules

Each user of the Restaurant Order Tracker needs to create an account with a distinct working email address and password. Invalid emails are prohibited through checks. Every user account belongs to a certain person and cannot be shared as each user will have their own unique id.

Users must choose between the Staff and Customer roles upon registration. Their rights and the features they can access within the app are specified by this role. Once registered, roles cannot be altered. Customers are unable to create or modify orders, and staff are unable to rate orders.

A staff member may make more than one order. Every order needs to include a timestamp, the restaurant name(which is automatically filled in from the database using the staff's id), and the customer's name. One consumer and one employee are associated with each order. Customers can place several orders at different restaurants.

Only one of the three statuses—Pending, Ready, or Collected—can be applied to an order at once. Pending is the default status for new orders. Order statuses can only be updated by employees allocated to a restaurant, and customers are instantly notified of these changes, which they can see on their dashboard.

Consumers have the ability to examine their whole order history and assign a thumbs up or down to each order that they have made. Ratings cannot be changed once they are submitted, and they are only allowed to rate each order once. Only the customer of the order may submit a rating, and each rating is associated with a particular user and order.

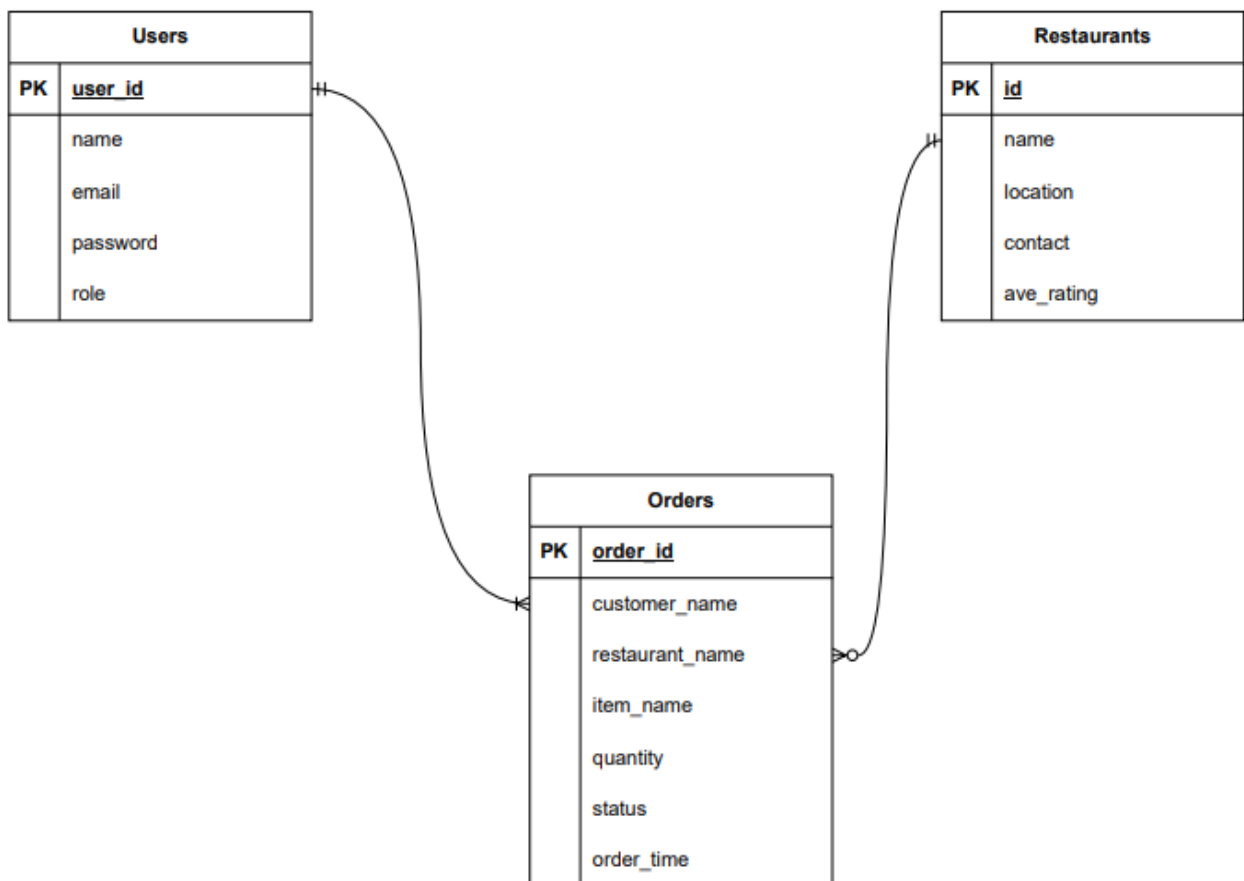
The average rating for each restaurant is determined by counting how many thumbs up and thumbs down it gets. From this, the average is calculated using the total amount of ratings that have been made. Binary ratings are kept in the Ratings table and are connected to the restaurant and the order.

Restaurants can receive ratings from multiple customers. A restaurant may have several staff members, but each staff member is associated with only one restaurant.

To maintain **data integrity**, no required fields (such as usernames, statuses, or ratings) can be left empty. Validation logic ensures completeness of data before submission.

App functionality is dynamically tailored to the logged-in user's role. All user activity is saved in a **centralized relational database**, ensuring consistent, reliable data management. The system supports multiple restaurants and users simultaneously without data conflict.

Initial ERD

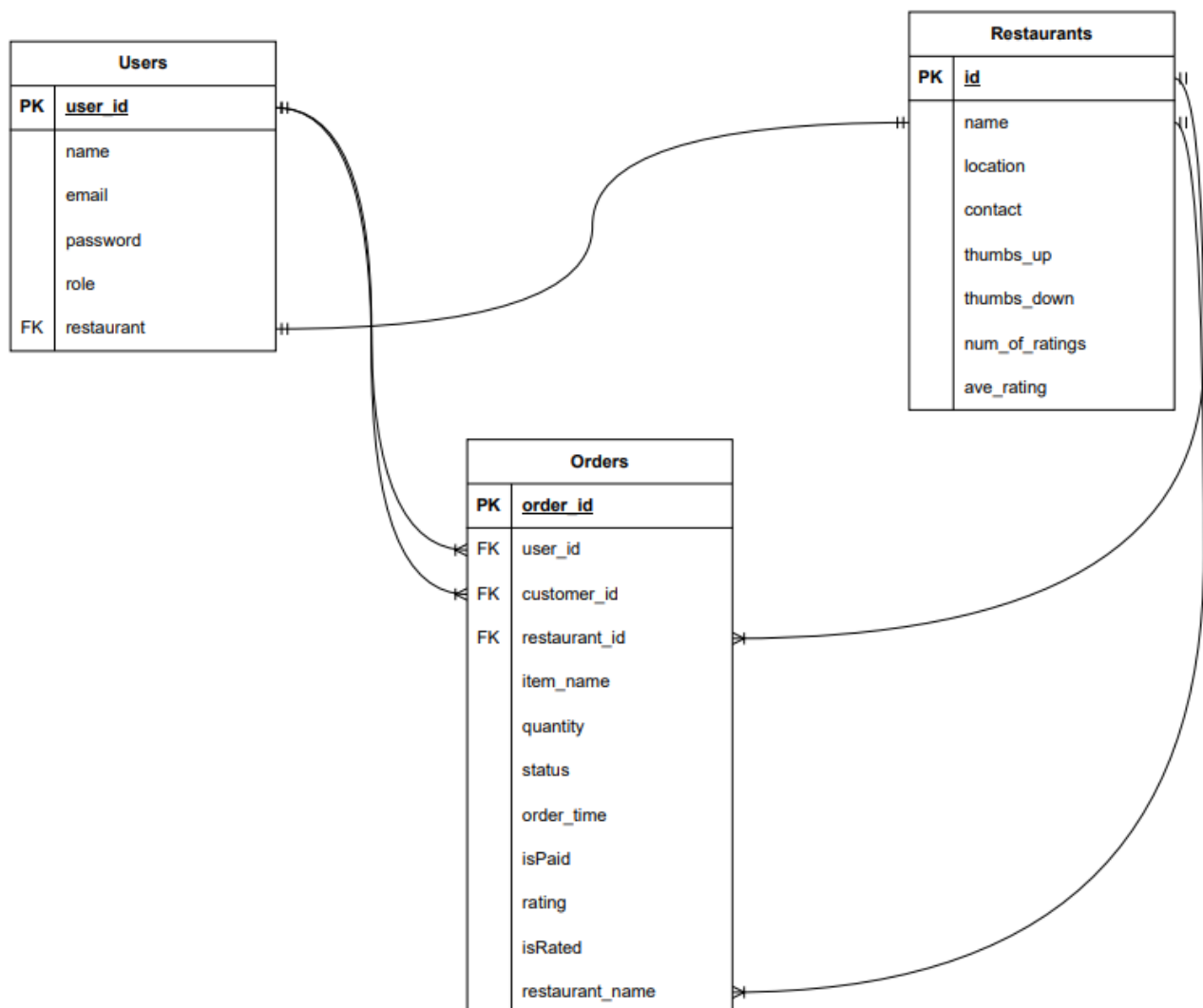


Issues with Initial ERD

- 1) **Lack of Proper Relationships:** The initial ERD fails to establish proper foreign key relationships between tables. The Orders table stores customer_name and restaurant_name as plain text instead of using foreign keys to link to the tables through their id's. This design leads to data redundancy and inconsistency, as the same customer or restaurant name might be spelled differently across orders. It also makes it impossible to enforce referential integrity, meaning orders without valid users or restaurants could exist.
- 2) **Improper Normalization:** The Restaurants table directly stores ave_rating as a computed field, violating normalization principles (specifically Third Normal Form). This creates redundancy and potential inconsistencies because the average rating should be derived from actual order ratings rather than stored separately. Additionally, the Orders table duplicates restaurant names instead of referencing the Restaurants table via a foreign key, which could lead to mismatches if a restaurant's name changes.
- 3) **Missing Critical Fields:** The initial ERD omits important fields like isPaid, rating, and isRated in the Orders table, which are necessary for tracking payment status and customer feedback. The Users table also lacks a restaurant field to associate staff members with specific restaurants, making it impossible to enforce role-based access control (e.g., ensuring only staff from the correct restaurant can manage its orders).
- 4) **No Role-Based Constraints:** While the Users table includes a role field, the schema lacks mechanisms to enforce role-specific rules. For example, there's no way to prevent a customer from being incorrectly assigned as a restaurant staff member or to ensure only customers can place orders. This logic would have to be handled entirely in the application layer, increasing the risk of data integrity issues if the application is bypassed.
- 5) **Poor Handling of Ratings:** The initial ERD doesn't include a proper rating system at all, missing fields like thumbs_up, thumbs_down, and num_of_ratings that are present in the final design. Without these, the system cannot track detailed feedback or prevent duplicate ratings, leading to potential inaccuracies in restaurant evaluations.

- 6) **The initial ERD lacks proper foreign key relationships**, storing customer_name and restaurant_name as raw text in Orders instead of linking to Users and Restaurants via IDs. This leads to broken referential integrity (allowing invalid references), data redundancy (inconsistent naming), missing role enforcement (no staff-restaurant assignment), and update anomalies (manual name changes required). Proper foreign keys (user_id, restaurant_id) are essential to enforce valid relationships and maintain data consistency.

Final ERD



Solutions

1. **Normalization and Foreign Key Implementation:**

The initial ERD stored `customer_name` and `restaurant_name` as plain text, causing redundancy and integrity issues. The final design resolves this by enforcing proper normalization and foreign key relationships. The `Orders` table now links to `Users` and `Restaurants` via `user_id` and `restaurant_id`, ensuring referential integrity. This eliminates duplicate data (e.g., inconsistent restaurant names) and automates updates—changing a restaurant’s name propagates seamlessly to all related orders.

2. **Validation and NOT NULL Constraints:**

Critical fields like `isPaid`, `rating`, and `restaurant` (for staff assignments) are now mandatory (NOT NULL), preventing incomplete records. Role-based validation ensures only customers can place orders and only staff can manage them, enforced at both the database and application levels.

3. **Dynamic Rating System:**

The final ERD replaces the static `ave_rating` with computed fields (`thumbs_up`, `thumbs_down`, `num_of_ratings`) derived from order ratings. A composite key (`user_id` + `order_id`) prevents duplicate ratings, while triggers automatically update restaurant averages, ensuring accuracy without redundancy.

4. **Role-Based Access Control:**

Staff are explicitly linked to restaurants via the `Users.restaurant` field, enabling strict permission checks. The schema now enforces role logic (e.g., customers cannot be assigned as staff), reducing reliance on application-layer validation.

5. **Triggers and Views for Enhanced Performance and Data Handling:**

To improve efficiency and data accuracy, the final design implements database triggers and views. A `RestaurantRatingView` dynamically calculates average ratings by aggregating live order feedback, replacing the static `ave_rating` field. Triggers automatically update `thumbs_up`, `thumbs_down`, and `num_of_ratings` whenever new ratings are submitted, ensuring real-time consistency. Additionally, an `OrderSummaryView` consolidates order details with user and restaurant information, simplifying reporting and reducing redundant queries. These optimizations eliminate manual updates while guaranteeing accurate, up-to-date analytics.

Implementation of Tables / Functions / Procedures / Views / Triggers

The database tables were designed to directly support the app's key functionalities for both staff and customers. Each table is linked to the relevant screens in the mobile application. The final design includes only three main tables: Users, Restaurants, and Orders. Below is a breakdown of each table's purpose, the logic applied to it, and placeholders where relevant screenshots or ER diagrams can be added.

Users Table

user_id	name	email	password	role	restaurant
33	hemesh	hparshotam11@gmail.com	\$2y\$10\$WjQHBU2.J.YgDZIUPD3GesLYliFHUFooA0hCpa/an7eY30KSceS2	customer	NULL
36	Cr7	cr7@outlook.com	\$2y\$10\$CihRfZkG55N0zu08p0j2z.HBU0FOUhtPLXK08cL4K20PubUnuciy0	staff	Pizza Palace
40	jai cust	check@gmail.com	\$2y\$10\$XsgnkHnHJXbzomw8WkrSBEDJPFrwngcZ7MslvipCZ6w/9tmE1rFi.	customer	NULL
41	Messiah	messiah@gmail.com	\$2y\$10\$CZ1HkLQbUKUL8KVJo9KtpuNNL.vICu3tZvBg8SMIqFQ.7C0bi0iGC	staff	McGrub
42	Nayan Makanjee	nm@gmail.com	\$2y\$10\$K8G1RffwEV59AQh7JedC30PYG9KwyRMoL1zw.eJknk1BRM7h2dC0e	staff	Peri Grillz
43	Nayan	nm@gmail.com	\$2y\$10\$G22GiVE9ncvutQh3Yw0HpO/O.bQeYUeFY0eB/3Ke40WmrMV0eSoy	customer	NULL
44	staff jai	staffjai@gmail.com	\$2y\$10\$0F9xps7jjZsQ6aEwp5hZZ.Nv0q18nhiQCFRf1S2u95xT7ecVrbnhi	staff	Burger House
45	staff test 1	stafftest1@gmail.com	\$2y\$10\$TGFH58rdVN03ZuxrIEaazeyIqITGxllvCVwMye0S1vVgQEp5JS1IG	staff	Hen House
46	staff test 2	stafftest2@gmail.com	\$2y\$10\$S3yMteF/06q7VVDjiWkN0qvcCnly0zzmhl2DD6TGJ0ljVlyhVskW	staff	McGrub

The Users table is primarily used during the **Sign Up** and **Login** processes. When a new user registers, their information (name, email, password, role, and if applicable, restaurant name) is stored in this table. Validation ensures that each email is unique and correctly formatted. The role column determines whether the user is a **staff member** or **customer**, which in turn controls access to different sections of the app.

For staff, the restaurant field links them to a specific restaurant. For customers, this field is left as NULL. This design supports staff-level filtering when adding or updating orders, ensuring they only see and interact with orders from their assigned restaurant.

Restaurants Table

id	name	location	contact	thumbs_up	thumbs_down	num_of_ratings	ave_rating
1	Burger House	12 Lion Street	0723456789	4	3	7	5.71
2	Sushi Corner	8 2nd Street	0834567890	3	1	4	7.50
3	Pizza Palace	45 Acacia Avenue	0745678901	0	5	5	0.00
4	McGrub	19 Zebra Street	0612345678	6	7	13	4.62
5	Hen House	33 7th Street	0829876543	2	5	7	2.86
6	Peri Grillz	27 Oak Street	0798765432	6	2	8	7.50

The Restaurants table contains static information about all takeaways supported by the app. It includes fields like restaurant name, location, contact details, logo image URL, thumbs up/down counts, number of ratings, and average rating. These values are dynamically updated as customers rate their orders. Staff cannot modify this data directly; it is updated via the logic linked to the Orders table and Rating inputs.

The average rating is **not manually entered**—it is calculated using a **trigger** when a customer submits a thumbs up or thumbs down. The app then reflects the updated average rating in the restaurant's profile view and the customer order history.

Orders Table

order_id	user_id	customer_id	restaurant_id	item_name	quantity	status	order_time	isPaid	rating	isRated	restaurant_name
1	35	NULL	6	jollof rice	5	pending	2025-05-25 11:22:35	0	NULL	0	Peri Grillz
2	35	NULL	6	dhal	18	pending	2025-05-25 11:27:05	0	NULL	0	Peri Grillz
3	36	NULL	3	chicken pizza	4	pending	2025-05-26 13:53:38	0	NULL	0	Pizza Palace
4	35	NULL	6	dhal and rice	18	pending	2025-05-26 18:00:33	0	NULL	0	Peri Grillz
5	35	NULL	6	888	1	pending	2025-05-26 18:01:33	0	NULL	0	Peri Grillz
6	35	NULL	6	ygsuhs	1	pending	2025-05-26 19:34:33	0	NULL	0	Peri Grillz
7	35	NULL	6	shrub	14	pending	2025-05-26 22:09:10	0	NULL	0	Peri Grillz
8	36	NULL	3	ur ma potatoes	2	pending	2025-05-27 14:41:22	0	NULL	0	Pizza Palace
9	32	25	2	Trophy	5	Collected	2025-05-27 19:15:30	1	0	1	Sushi Corner
10	36	17	3	lows	2	Collected	2025-05-27 19:20:18	1	0	1	Pizza Palace
11	35	33	6	thgbj	8	Ready	2025-05-27 19:20:31	0	NULL	0	Peri Grillz

The Orders table holds all information about customer orders. It is populated when a staff member creates a new order via the **Create Order** page. Required fields include:

- user_id (staff who created the order),
- customer_id (linked to the customer who will receive it),
- restaurant_id (to identify the source restaurant),
- item_name, quantity, and order_time.

Each order has a status field (Pending, Ready, or Collected), a rating value (0 = thumbs down, 1 = thumbs up, NULL if not yet rated), and an isPaid flag. A boolean isRated field tracks whether the customer has already rated that order, preventing duplicate ratings.

To ensure consistent data:

- **Validation** is used to prevent empty fields for required values.
- **Foreign keys** link each order to valid users and restaurants.
- **Composite logic** ensures that a customer can't rate the same order more than once.

Functions and Procedures

- **AddOrder(user_id, customer_id, restaurant_id, item_name, quantity)**
Used by staff to insert a new order into the system.
- **UpdateOrderStatus(order_id, new_status)**
Called when staff update an order's status from Pending → Ready → Collected.
- **SubmitRating(order_id, rating_value)**
Called when a customer rates their order with a thumbs up (1) or thumbs down (0). Updates the rating, isRated, and triggers recalculation of the restaurant's average.

Views

- **CustomerOrderHistoryView**
Displays all past and current orders for a logged-in customer, including order status, restaurant name, and rating.
- **RestaurantAverageRatingsView**
Aggregates thumbs up/down across all orders and calculates the live average rating per restaurant. This view supports the restaurant information pages for both customers and shows the rating of the restaurant on the staff menu.

Triggers

- Once a customer rates and order, the restaurants table automatically updates the thumbs up/down counters and recalculates the average rating for the associated restaurant when a new rating is submitted. This trigger ensures that the average is always up-to-date without requiring manual recalculation.
- The staff can change the status of a customers order, which is automatically stored in the database and then indicates the status of the order on the customers dashboard. Once the status is changed to be collected, it cannot be changed to be pending or ready again.

Normalization

Normalization is the process of organizing data in a database to reduce redundancy and improve data integrity. For the **Restaurant Order Tracker** project, normalization was applied from the early stages of design. Initial drafts of the ERD included fewer tables with many attributes, which resulted in data repetition and complex dependencies. These were refined through normalization into a cleaner, more structured schema.

Through this process, data that was initially stored as plain text—such as restaurant names and user details—was moved into separate, dedicated tables (Users, Restaurants) and referenced via **foreign keys** in the Orders and Ratings tables. This change improved data consistency and made updates easier to manage.

Normalization was applied up to **Second Normal Form (2NF)**:

- **1NF**: Ensured all fields hold atomic (single) values.
- **2NF**: Separated data into logical tables, ensuring that fields depend on the entire primary key.

Additional **validation logic** and **foreign key constraints** were used to ensure that no incomplete or invalid data could be inserted into the system. Relationships between users, orders, and restaurants were clearly defined.

As a result, the final database structure is efficient, easy to maintain, and less prone to data inconsistencies. Upon further retrospection, there was a missed opportunity to apply Third Normal Form (3NF) to the database in order to make it even more efficient and safe from inconsistency as that would remove the transitive dependencies. These are ultimately insights and key takeaways that we can learn from.

Improvements and Retrospection

This project has taught us about the many components that goes into making an app and having it work with a database. There have been many takeaways from this from which we can learn and improve upon in the future.

The database is one of the most important elements in designing an app of this standard. As said above in the normalization segment, there was a missed opportunity to improve the tables in terms of data redundancy and integrity. Although the tables were made before the process of coding in Android Studio, there were changes made to them as we progressed through the project.

As a result, the database had a drawback in normalization. We also learnt that it is safer to ensure that most if not all key elements in the app should be run through the backend. This allows for easier change if need be and allows for futureproofing of the project.

We have made the user experience to be as easy as possible as can be and to be very intuitive for both the staff side and the customer side. This keeps the whole process simpler on both ends, but still effective enough for all the functions of the app to fulfil their needs.

Overall, the project was a massive learning experience and provided a lot of insight into the many modules and stages that take place in the design and implementation processes. Throughout the challenges that would come up and be required to fix, we persevered and worked toward our final goal.